

Control Systems Primer

A Guide to Understanding the STAR Robot Control Architecture

STAR: Autonomous Mapping Robot

ESET 462 – Control Systems

Texas A&M University

Fall 2025

Team Members:

Elizabeth AJose Abhiraj Lakkamraju Jacob Lopez
Jackson Lopez Cesar Magana Ryan Ingram Nafi Islam

Read this document to understand the control systems architecture of the STAR robot!

Contents

1	System Overview	2
1.1	What is the STAR Robot?	2
1.2	System Topology	2
1.3	Control System Hierarchy	2
2	Cascaded PID Control Architecture	4
2.1	Why Cascaded Control?	4
2.2	The Three Control Loops	4
2.2.1	Position Loop (Outer Loop)	4
2.2.2	Velocity Loop (Middle Loop)	4
2.2.3	Current Loop (Inner Loop)	4
2.3	Transfer Function Analysis	5
2.3.1	Motor Electrical Model	5
2.3.2	Motor Mechanical Model	5
2.4	PID Tuning Approach	6
3	Sensor Integration	7
3.1	Sensor Overview	7
3.2	BNO055 IMU Integration	7
3.3	Temperature-Compensated Distance Measurement	7
3.4	Current Sensing with DRV8243-Q1	8
4	Supervisory Safety System	9
4.1	Multi-Level Safety Architecture	9
4.2	Safety Conditions	9
4.3	State Machine	9
4.4	Soft Start Implementation	10
5	SLAM and Navigation Layer	11
5.1	Hector-SLAM Overview	11
5.2	Nav2 Path Planning	11
5.3	SPI Communication Protocol	11
6	Stability Analysis	13
6.1	Loop Stability Requirements	13
6.2	Bode Plot Analysis	13
6.3	Closed-Loop Poles	13
6.4	Robustness Considerations	13
7	Discrete-Time Implementation	15
7.1	Sampling Considerations	15
7.2	Discrete PID Implementation	15
7.3	Anti-Windup Implementation	15
7.4	Code Structure	16

8 Hardware Specifications	17
8.1 Motor System	17
8.2 Motor Driver (DRV8243-Q1)	17
8.3 Power System	17
8.4 Processor Specifications	18
Appendix A: Glossary	19
Appendix B: Schemabloc Reference	20
Appendix C: Quick Reference	21

1 System Overview

1.1 What is the STAR Robot?

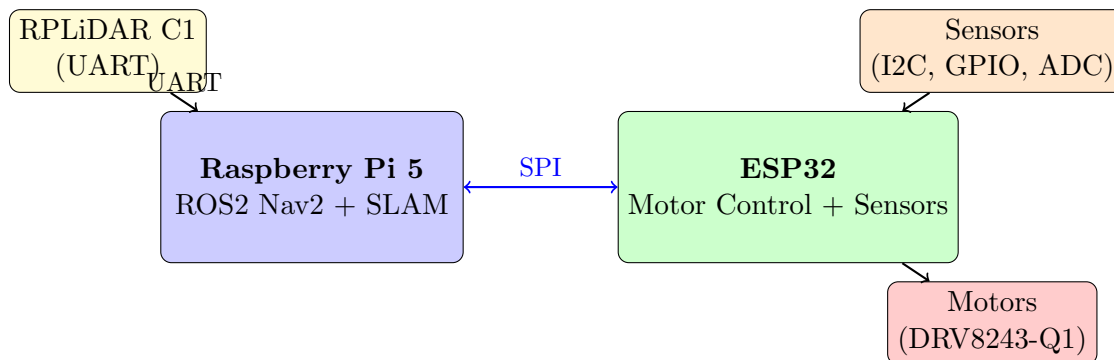
The STAR (Simultaneous Localization and Mapping) robot is an autonomous mobile platform designed to map indoor environments. It uses a sophisticated control system architecture with two main processing layers:

1. **Navigation Layer (Raspberry Pi 5):** Runs ROS2 Nav2 with Hector-SLAM for mapping and path planning
2. **Motor Control Layer (ESP32):** Executes cascaded PID control for precise motor control at 50Hz

In Plain English

Think of it like a self-driving car in miniature. The Raspberry Pi is the “brain” that decides where to go (using SLAM and path planning), while the ESP32 is the “reflexes” that make the wheels turn smoothly and safely.

1.2 System Topology



1.3 Control System Hierarchy

Our robot uses a **hierarchical control architecture**:

Level	Processor	Function	Rate
Planning	RPi5	Global path planning (A* algorithm)	1 Hz
Navigation	RPi5	Local planning (DWA)	10 Hz
Velocity	ESP32	Velocity setpoint tracking	50 Hz
Current	ESP32	Motor current regulation	50 Hz
Safety	ESP32	Supervisory override	50 Hz

Table 1: Control hierarchy from high-level planning to low-level actuation

Why This Matters

This hierarchical approach separates concerns: the RPi5 handles computationally expensive SLAM and planning, while the ESP32 handles time-critical motor control. This ensures smooth motion even when the Pi is busy processing sensor data.

2 Cascaded PID Control Architecture

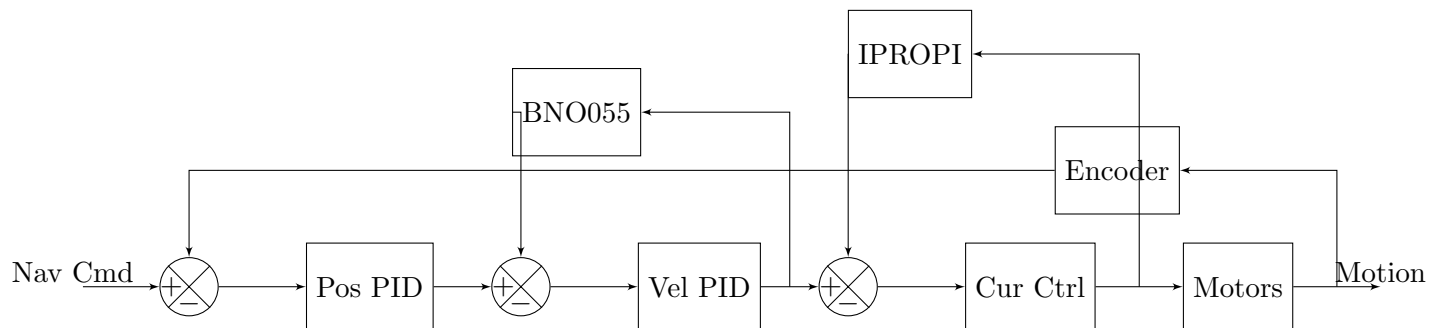
2.1 Why Cascaded Control?

A single PID controller works well for simple systems, but our robot needs precise control of both position and velocity while protecting the motors from overcurrent. We achieve this with **cascaded PID loops**.

In Plain English

Imagine a thermostat that not only controls room temperature but also monitors how fast the furnace is heating and how much fuel it's using. Each “loop” handles one aspect, and they work together like nested layers of control.

2.2 The Three Control Loops



2.2.1 Position Loop (Outer Loop)

- **Setpoint:** Desired position from ROS2 Nav2
- **Feedback:** Quadrature encoders on each motor
- **Output:** Velocity setpoint for the velocity loop
- **Purpose:** Ensure the robot reaches the commanded position

2.2.2 Velocity Loop (Middle Loop)

- **Setpoint:** Velocity command from position loop
- **Feedback:** BNO055 IMU linear acceleration (integrated) + encoder derivative
- **Output:** Current setpoint for the current loop
- **Purpose:** Maintain smooth, consistent wheel speed

2.2.3 Current Loop (Inner Loop)

- **Setpoint:** Current command from velocity loop
- **Feedback:** DRV8243-Q1 IPROPI analog current sense

- **Output:** PWM duty cycle to motor driver
- **Purpose:** Protect motors from overcurrent, ensure torque control

Key Point

Loop Bandwidth Rule: Each inner loop must be 5-10x faster than its outer loop. Our current loop runs at the full 50Hz, velocity loop effectively at 50Hz with filtered feedback, and position loop commands come at 10Hz from Nav2.

2.3 Transfer Function Analysis

Each loop can be modeled with its own transfer function:

2.3.1 Motor Electrical Model

The DC motor armature circuit is modeled as:

$$G_{motor}(s) = \frac{I(s)}{V(s)} = \frac{1}{Ls + R} \quad (1)$$

Where:

- L = Motor inductance (0.5 mH typical)
- R = Motor resistance (1.5 Ω typical)

2.3.2 Motor Mechanical Model

The mechanical dynamics:

$$G_{mech}(s) = \frac{\omega(s)}{I(s)} = \frac{K_t}{Js + b} \quad (2)$$

Where:

- K_t = Torque constant (0.01 N m A⁻¹)
- J = Rotor inertia (1×10^{-5} kg m²)
- b = Viscous friction (1×10^{-4} N m s)

STAR Project Values

Estimated Plant Parameters:

- Motor resistance: $R = 1.5 \Omega$
- Motor inductance: $L = 0.5$ mH
- Torque constant: $K_t = 0.01$ N m A⁻¹
- Electrical time constant: $\tau_e = L/R = 0.33$ ms
- Mechanical time constant: $\tau_m = J/b = 0.1$ s

2.4 PID Tuning Approach

We use a **sequential tuning** approach:

1. **Tune current loop first** (innermost, fastest)
 - High bandwidth for fast current response
 - Primarily P control with small I for offset correction
2. **Tune velocity loop second**
 - Moderate bandwidth (5-10x slower than current loop)
 - PI control for zero steady-state velocity error
3. **Tune position loop last** (outermost, slowest)
 - Lowest bandwidth to prevent oscillation
 - Often P-only or PI with very small integral gain

STAR Project Values

Initial PID Gains (to be tuned):

- **Current Loop:** $K_p = 5.0$, $K_i = 100$, $K_d = 0$
- **Velocity Loop:** $K_p = 2.0$, $K_i = 0.5$, $K_d = 0.05$
- **Position Loop:** $K_p = 1.0$, $K_i = 0.1$, $K_d = 0$

3 Sensor Integration

3.1 Sensor Overview

The STAR robot uses multiple sensors for feedback and safety:

Sensor	Interface	Purpose	Processor
RPLiDAR C1	UART	SLAM mapping	RPi5
BNO055 IMU	I2C	Orientation, velocity	ESP32
DS18B20+	OneWire	Temperature compensation	ESP32
HC-SR04	GPIO	Proximity safety	ESP32
DRV8243 IPROPI	ADC	Current sensing	ESP32
Quadrature Encoders	GPIO	Position feedback	ESP32
BQ78350 Fuel Gauge	SMBus	Battery monitoring	ESP32

Table 2: Sensor assignments and interfaces

3.2 BNO055 IMU Integration

The BNO055 provides critical feedback for the velocity control loop:



BNO055 Outputs Used:

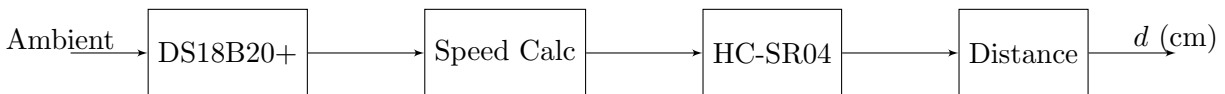
- **Orientation (Euler angles):** Heading feedback for heading PID
- **Linear acceleration:** Velocity estimation via integration
- **Gyroscope:** Angular velocity for turn rate feedback

Key Point

The BNO055 has an onboard sensor fusion processor that combines accelerometer, gyroscope, and magnetometer data. This offloads computation from the ESP32 and provides drift-corrected orientation.

3.3 Temperature-Compensated Distance Measurement

The HC-SR04 ultrasonic sensor's accuracy depends on the speed of sound, which varies with temperature. We use the DS18B20+ for compensation:



Temperature Compensation Formula:

$$v_{sound} = 331.3 + 0.606 \cdot T \quad (3)$$

Where T is temperature in Celsius and v_{sound} is in m/s.

Distance Calculation:

$$d = \frac{v_{sound} \cdot t_{echo}}{2} \quad (4)$$

In Plain English

At room temperature (20 °C), sound travels at 343 ms⁻¹. At 0 °C, it's only 331 ms⁻¹. Without compensation, a 10cm measurement could be off by 3-4mm—significant for close-range safety!

3.4 Current Sensing with DRV8243-Q1

The DRV8243-Q1 motor driver includes an IPROPI (current sense) output:

**IPROPI Characteristics:**

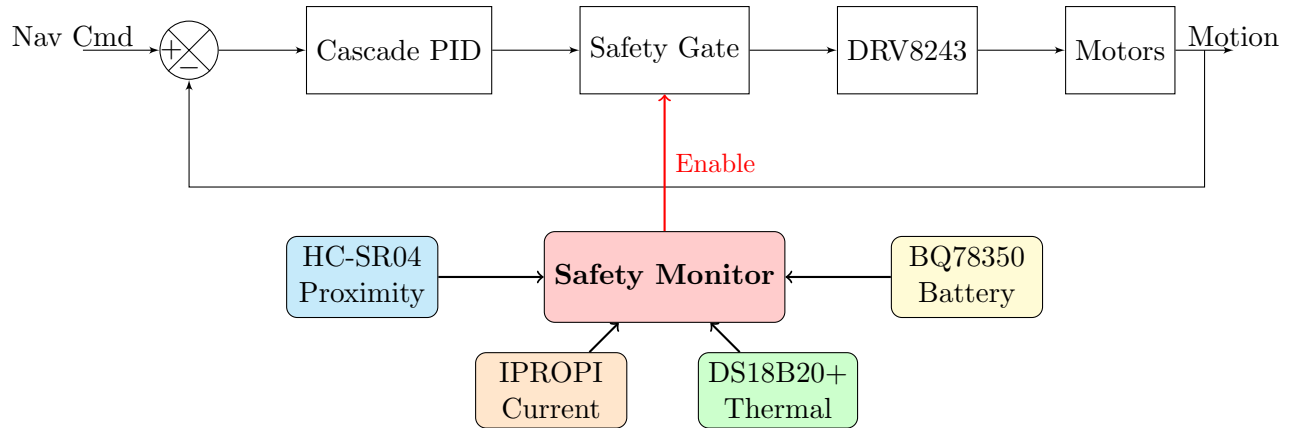
- Output: Analog voltage proportional to motor current
- Gain: 1100 $\mu\text{A A}^{-1}$ (typical)
- With 1 k Ω sense resistor: 1.1 mV A^{-1}
- ESP32 ADC: 12-bit, 0 V to 3.3 V range

$$I_{motor} = \frac{V_{ADC}}{R_{sense} \cdot A_{IPROPI}} = \frac{V_{ADC}}{1000 \cdot 1100 \times 10^{-6}} \quad (5)$$

4 Supervisory Safety System

4.1 Multi-Level Safety Architecture

The STAR robot implements a **supervisory control layer** that can override the PID controllers for safety:



4.2 Safety Conditions

The Safety Gate implements the following conditions:

Condition	Threshold	Action	Recovery
Proximity	$d < 10 \text{ cm}$	Stop motors	$d > 15 \text{ cm}$
Overcurrent	$I > 8 \text{ A}$	Reduce PWM	$I < 6 \text{ A}$
Thermal	$T > 60^\circ\text{C}$	Reduce power	$T < 50^\circ\text{C}$
Low Battery	$V < 9.6 \text{ V}$	Safe shutdown	Manual reset
Stall Detect	$I > 5 \text{ A}, \omega = 0$	Stop motor	Manual reset

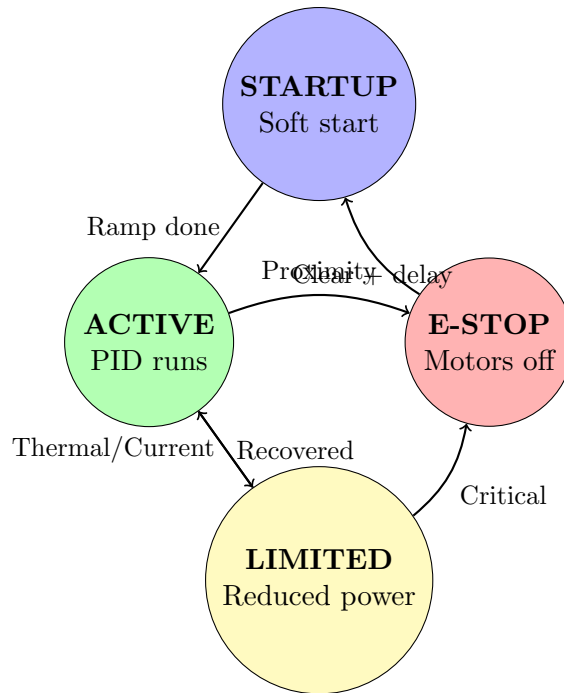
Table 3: Safety conditions with hysteresis thresholds

Safety Critical

All safety thresholds include **hysteresis** to prevent oscillation. For example, proximity detection turns off at 10cm but requires 15cm clearance to resume—this 5cm gap prevents flickering when objects are near the threshold.

4.3 State Machine

The safety system operates as a state machine:



4.4 Soft Start Implementation

When transitioning from E-STOP to ACTIVE, we use a **soft start** ramp:

$$PWM(t) = PWM_{target} \cdot \left(3 \left(\frac{t}{T_{ramp}} \right)^2 - 2 \left(\frac{t}{T_{ramp}} \right)^3 \right) \quad (6)$$

This “smoothstep” function provides:

- Zero velocity at start and end of ramp
- Smooth acceleration (no jerks)
- $T_{ramp} = 1$ s default duration

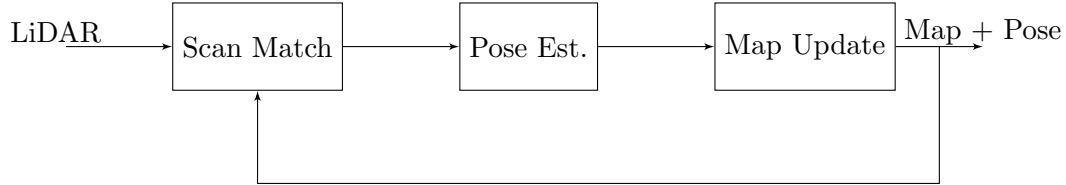
Why This Matters

Soft start prevents current spikes that could trip the overcurrent protection immediately after resuming. It also resets the PID integrators to prevent “windup” from the stopped period.

5 SLAM and Navigation Layer

5.1 Hector-SLAM Overview

The Raspberry Pi 5 runs **Hector-SLAM** for simultaneous localization and mapping:

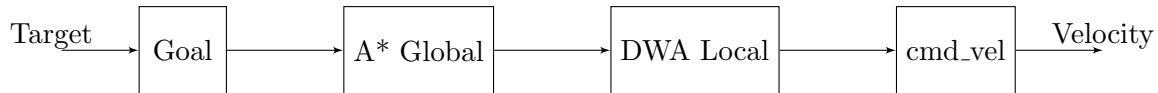


Hector-SLAM Advantages:

- No odometry required (uses scan-matching only)
- Works well with high-frequency LiDAR (15Hz RPLiDAR C1)
- Real-time capable on RPi5

5.2 Nav2 Path Planning

ROS2 Nav2 provides the navigation stack:



Planning Layers:

1. **Global Planner (A*)**: Finds optimal path on occupancy grid (1 Hz)
2. **Local Planner (DWA)**: Dynamic Window Approach for obstacle avoidance (10 Hz)
3. **Velocity Smoother**: Ensures commands respect acceleration limits

5.3 SPI Communication Protocol

The RPi5 sends velocity commands to ESP32 via SPI:

Byte	Field	Type	Description
0	Header	uint8	0xAA (start byte)
1-2	Linear X	int16	Linear velocity (mm/s)
3-4	Linear Y	int16	Lateral velocity (mm/s)
5-6	Angular Z	int16	Angular velocity (mrad/s)
7	Checksum	uint8	XOR of bytes 1-6

Table 4: SPI command packet structure (8 bytes)

STAR Project Values**Communication Parameters:**

- SPI Clock: 1 MHz
- Mode: SPI Mode 0 (CPOL=0, CPHA=0)
- Command Rate: 50 Hz (matches control loop)
- Timeout: 100 ms (triggers E-STOP if exceeded)

6 Stability Analysis

6.1 Loop Stability Requirements

For cascaded control, each loop must be stable independently:

Key Point

Stability Rule for Cascaded Loops:

1. Inner loops must be faster than outer loops (bandwidth separation)
2. Each loop must have positive gain and phase margins
3. Typically: $\omega_{inner} > 5 \times \omega_{outer}$

6.2 Bode Plot Analysis

For the velocity loop with plant $G_v(s) = \frac{K_m}{(\tau_m s + 1)}$:

$$G_v(s) = \frac{10}{0.1s + 1} \quad (7)$$

With PI controller $C_v(s) = K_p + \frac{K_i}{s} = 2.0 + \frac{0.5}{s}$:

Open-loop transfer function:

$$L_v(s) = C_v(s) \cdot G_v(s) = \frac{(2.0s + 0.5) \cdot 10}{s(0.1s + 1)} \quad (8)$$

6.3 Closed-Loop Poles

The closed-loop transfer function:

$$T(s) = \frac{L(s)}{1 + L(s)} \quad (9)$$

For stability, all poles must have negative real parts (left half-plane).

STAR Project Values

Expected Stability Margins:

- Gain Margin: > 10 dB (target)
- Phase Margin: $> 45^\circ$ (target)
- Crossover Frequency: 10 rad s^{-1} (velocity loop)

6.4 Robustness Considerations

The system must remain stable under:

- Parameter variations (motor wear, temperature)
- Load disturbances (terrain changes)
- Sensor noise (ADC quantization, EMI)

- Communication delays (SPI latency)

Why This Matters

We design for “robust stability”—the system should work even when parameters change by $\pm 20\%$ from nominal. This is why we target generous stability margins rather than optimal performance.

7 Discrete-Time Implementation

7.1 Sampling Considerations

The ESP32 runs the control loop at 50 Hz ($T_s = 20$ ms):

$$f_s = 50 \text{ Hz}, \quad T_s = 20 \text{ ms} \quad (10)$$

Nyquist Criterion:

$$f_s > 2 \cdot f_{\text{bandwidth}} \quad (11)$$

With velocity loop bandwidth of approximately 10 rad s^{-1} (1.6 Hz):

$$50 \text{ Hz} > 2 \times 1.6 \text{ Hz} = 3.2 \text{ Hz} \quad \checkmark \quad (12)$$

We have approximately 15x oversampling, which is excellent.

7.2 Discrete PID Implementation

Using Tustin (bilinear) transformation:

$$P[n] = K_p \cdot e[n] \quad (13)$$

$$I[n] = I[n-1] + K_i \cdot T_s \cdot \frac{e[n] + e[n-1]}{2} \quad (14)$$

$$D[n] = \alpha \cdot K_d \cdot \frac{e[n] - e[n-1]}{T_s} + (1 - \alpha) \cdot D[n-1] \quad (15)$$

$$u[n] = P[n] + I[n] + D[n] \quad (16)$$

Where $\alpha = 0.1$ is the derivative filter coefficient.

7.3 Anti-Windup Implementation

To prevent integral windup during saturation:

```
// Conditional integration
if (output_saturated) {
    // Don't accumulate integral when output is limited
    integral_term = integral_term; // Hold
} else {
    integral_term += Ki * Ts * (error + prev_error) / 2.0;
}

// Clamp integral term
integral_term = constrain(integral_term, -I_MAX, I_MAX);
```

7.4 Code Structure

```
void control_loop_50Hz() {  
    // 1. Read sensors  
    float pos = read_encoders();  
    float vel = read_bno055_velocity();  
    float current = read_ipropi();  
  
    // 2. Check safety conditions  
    if (check_safety_conditions() != SAFE) {  
        enter_estop();  
        return;  
    }  
  
    // 3. Position loop (outer)  
    float vel_setpoint = position_pid(pos_setpoint, pos);  
  
    // 4. Velocity loop (middle)  
    float cur_setpoint = velocity_pid(vel_setpoint, vel);  
  
    // 5. Current loop (inner)  
    float pwm = current_pid(cur_setpoint, current);  
  
    // 6. Apply output  
    set_motor_pwm(pwm);  
}
```

8 Hardware Specifications

8.1 Motor System

Parameter	Value
Motor Type	DC Brushed with Encoder
Voltage	12 V nominal
Stall Current	3 A
No-Load Speed	500 RPM
Encoder Resolution	500 PPR (2000 CPR quadrature)
Quantity	4 (differential drive, 4WD)

Table 5: Motor specifications

8.2 Motor Driver (DRV8243-Q1)

Parameter	Value
Configuration	Dual H-Bridge
Continuous Current	12 A per channel
Peak Current	24 A
PWM Frequency	20 kHz
Current Sense	I ² C analog output
Protection	Overcurrent, thermal shutdown
Quantity	2 (one per motor pair)

Table 6: DRV8243-Q1 specifications

8.3 Power System

Rail	Voltage	Current Capacity
Battery	11.1 V (3S LiPo)	5000 mA h
Motor Power	12 V (regulated)	24 A max
Logic 5V	5 V	3 A
Logic 3.3V	3.3 V	500 mA

Table 7: Power distribution

8.4 Processor Specifications

Parameter	Raspberry Pi 5	ESP32
CPU	Cortex-A76 Quad @ 2.4 GHz	Xtensa Dual @ 240 MHz
RAM	4 GB	520 kB
Role	SLAM, Navigation	Motor Control
OS	Ubuntu 22.04 + ROS2	FreeRTOS
Control Rate	10 Hz (Nav2)	50 Hz (PID)

Table 8: Processing units comparison

STAR Project Values

Total System Budget:

- Raspberry Pi 5 (4GB): \$60
- ESP32 DevKit: \$10
- DRV8243-Q1 (x2): \$15
- Motors with encoders (x4): \$80
- RPLiDAR C1: \$100
- BNO055 IMU: \$30
- Sensors and passives: \$25
- Power system: \$50
- Chassis and hardware: \$80
- **Total:** Approximately \$450

Appendix A: Glossary

Term	Definition
Cascaded Control	Multiple nested control loops where the output of an outer loop is the setpoint for an inner loop
DWA	Dynamic Window Approach—local path planning algorithm
Hector-SLAM	Scan-matching SLAM algorithm that doesn't require odometry
IPROPI	Current sense output pin on DRV8243-Q1 motor driver
Nav2	ROS2 Navigation Stack for autonomous mobile robots
Nyquist Rate	Minimum sampling rate (2x bandwidth) to avoid aliasing
SLAM	Simultaneous Localization and Mapping
Soft Start	Gradual ramp-up of motor power to prevent current spikes
Supervisory Control	Higher-level control layer that can override lower loops
Tustin Transform	Bilinear transformation for converting continuous to discrete

Appendix B: Schemabloc Quick Reference

Command	Description
<code>\sbEntree{name}</code>	Create input node
<code>\sbSortie[dist]{name}{ref}</code>	Create output node
<code>\sbBloc[dist]{name}{label}{ref}</code>	Create processing block
<code>\sbComp{name}{ref}</code>	Create comparator/summer
<code>\sbRelier[label]{from}{to}</code>	Connect nodes
<code>\sbRenvoi{from}{to}{label}</code>	Create feedback path
<code>\sbNomLien[pos]{ref}{text}</code>	Label a connection
<code>\sbBlocr{name}{label}{ref}</code>	Feedback block (reversed)
<code>\sbRelieryx{from}{to}</code>	Vertical-then-horizontal path
<code>\sbRelierxy{from}{to}</code>	Horizontal-then-vertical path

Table 9: Essential schemabloc commands

Basic Closed-Loop Example:

```

\begin{tikzpicture}
  \sbEntree{E}
  \sbComp{sum}{E}
  \sbBloc[3]{ctrl}{Controller}{sum}
  \sbBloc[3]{plant}{Plant}{ctrl}
  \sbSortie[3]{S}{plant}
  \sbRelier{E}{sum}
  \sbRelier{sum}{ctrl}
  \sbRelier{ctrl}{plant}
  \sbRelier{plant}{S}
  \sbRenvoi{plant-S}{sum}{}
  \sbNomLien[0.5]{E}{Reference}
  \sbNomLien[0.5]{S}{Output}
\end{tikzpicture}

```

Appendix C: Demo Quick Reference

Parameter	Value
Control Loop Rates	
Position Loop	10 Hz (from Nav2)
Velocity Loop	50 Hz
Current Loop	50 Hz
Safety Monitor	50 Hz
Safety Thresholds	
Proximity Stop	< 10 cm
Proximity Resume	> 15 cm
Overcurrent Limit	> 8 A
Thermal Limit	> 60 °C
Low Battery	< 9.6 V
Communication	
SPI Clock	1 MHz
Command Packet	8 bytes
Timeout	100 ms
PID Gains (Initial)	
Position: K_p, K_i	1.0, 0.1
Velocity: K_p, K_i, K_d	2.0, 0.5, 0.05
Current: K_p, K_i	5.0, 100

Table 10: Quick reference values for demo

Common Demo Questions

Q: Why two processors?

A: The RPi5 handles computationally expensive SLAM (scan matching, map updates) while the ESP32 runs time-critical motor control at a guaranteed 50Hz. This separation ensures smooth motion even during heavy mapping computation.

Q: Why cascaded PID instead of single loop?

A: Cascaded control lets us protect the motors (current limit), maintain smooth speed (velocity loop), and reach positions accurately (position loop)—all independently tunable.

Q: What happens if communication fails?

A: The ESP32 has a 100ms timeout. If no SPI commands arrive, it triggers E-STOP and stops all motors. The robot will remain stopped until communication is restored and manually reset.

Q: How does temperature affect the ultrasonic sensor?

A: Sound speed varies with temperature (0.6 ms^{-1} per degree C). Without compensation, measurements could be off by several millimeters. The DS18B20+ provides real-time temperature for correction.